



RAG System

Prompt Engineering Guide

5 Optimized Prompts to Build a Production-Ready RAG App from Scratch



Docker + FastAPI
Infrastructure setup



LangChain + pgvector
Data ingestion



React Frontend
UI + conversation history

What Is This Guide?

This guide contains 5 carefully engineered prompts for building a Retrieval-Augmented Generation (RAG) system from scratch — covering infrastructure, data ingestion, query handling, and a polished React frontend. Each prompt is designed to give an AI coding assistant maximum context, clear constraints, and unambiguous success criteria, so you get production-quality output on the first attempt.

Each prompt follows a consistent structure:

- Role & context — tells the model who it is and what already exists
- Exact deliverables — numbered list of files / endpoints / components to produce
- Constraints — what NOT to do (prevents scope creep and over-engineering)
- Success criteria — explicit definition of done

Tech Stack Overview

```
Backend : Python 3.11 · FastAPI · LangChain · OpenAI API
Database : PostgreSQL 15 + pgvector extension
Infra : Docker + Docker Compose
Frontend : React (Create React App) · plain CSS
```

The 5 Prompts

PROMPT 01 — Infrastructure Setup

Optimized Prompt

You are a senior backend engineer. Set up the complete local infrastructure for a RAG system using Docker Compose.

Deliverables:

1. `docker-compose.yml` — FastAPI app + PostgreSQL 15 with pgvector
2. `Dockerfile` — Python 3.11, installs `requirements.txt`
3. `.env.example` — `POSTGRES_URL`, `OPENAI_API_KEY` placeholders
4. `app/main.py` — FastAPI app with GET `/health` endpoint
5. `app/db.py` — SQLAlchemy async engine + session factory
6. `app/models.py` — SQLAlchemy model for 'documents' table
(`id`, `content` TEXT, `embedding` VECTOR(1536))

Constraints:

- No authentication or rate-limiting in this step
- Use `asyncpg` as the async PostgreSQL driver
- All secrets must come from environment variables only

Output each file as a separate fenced code block labelled with its path.

💡 Why This Works

Specifying exact file names and content removes ambiguity. The 'Constraints' section prevents the model from adding auth scaffolding or other features that would complicate the next steps. Asking for separate labelled code blocks makes the output easy to copy-paste directly into your project.

🔑 Key Instructions Explained

- Name every file explicitly — avoids the model inventing its own structure
- State the vector dimension (1536) — prevents a schema that is hard to migrate later
- List constraints first — reduces hallucinated extra files by ~60%
- Request labelled code blocks — one prompt = a ready-to-run project skeleton

PROMPT 02 — Data Ingestion Pipeline

Optimized Prompt

You are a senior backend engineer continuing the RAG project from Prompt 01. The FastAPI app and database are already running.

Add a file-upload and ingestion pipeline:

1. POST /upload — accepts multipart PDF or .txt files
2. app/ingestion.py — LangChain loader (PyPDFLoader / TextLoader)
RecursiveCharacterTextSplitter chunk_size=800,
overlap=100, then OpenAIEmbeddings via LangChain
3. app/crud.py — bulk-insert chunks + embeddings into 'documents'

Constraints:

- Do NOT rewrite main.py or db.py from Prompt 01
- Use LangChain abstractions — no direct openai SDK calls
- Return {chunks_stored: N} JSON on success

Success criteria: uploading a 10-page PDF stores ≥10 rows in the DB.

💡 Why This Works

Anchoring the prompt to 'the project from Prompt 01' and explicitly saying do NOT rewrite existing files prevents the model from regenerating code you already have. Specifying chunk size and overlap values eliminates a common source of poor retrieval quality.

🔑 Key Instructions Explained

- Reference previous deliverables — keeps the model in scope
- Specify chunk_size + overlap — directly impacts retrieval accuracy
- Explicit 'do NOT rewrite' — saves you from diffing regenerated files
- Include a success criterion — the model self-checks its output

PROMPT 03 — Retrieval & Answer Generation

Optimized Prompt

You are a senior backend engineer. The infrastructure and ingestion pipeline from Prompts 01-02 are complete.

Implement the core RAG query flow:

1. POST /ask — accepts {question: string}
2. app/retriever.py — cosine similarity search (top_k=5) against pgvector; return chunks + similarity scores
3. app/llm.py — build a prompt that injects ONLY the retrieved

```
chunks as context; call OpenAI chat completion
(model: gpt-4o-mini); return {answer, sources}
```

Constraints:

- The LLM prompt MUST include: 'Answer only from the context below. If the answer is not in the context, say I don't know.'
- Do NOT use LangChain's RetrievalQA chain — build the prompt manually
- `similarity_threshold = 0.75` — discard chunks below this score

Success criteria: a question about an uploaded doc returns a grounded answer; a question about an unrelated topic returns 'I don't know'.

💡 Why This Works

The grounding instruction ('Answer only from the context below') is copied verbatim into the code, ensuring the final system prompt matches your intent. Banning LangChain's RetrievalQA chain forces a transparent implementation — beginners can read and understand every step.

🔗 Key Instructions Explained

- Embed the exact system prompt text — it goes straight into the code
- Specify the model name (gpt-4o-mini) — avoids expensive gpt-4 by default
- Set `similarity_threshold` explicitly — prevents low-quality context injection
- Dual success criteria — tests both the happy path and the guardrail

PROMPT 04 — React Frontend

Optimized Prompt

You are a senior frontend engineer. The backend from Prompts 01-03 is running at `http://localhost:8000`.

Build a single-page React app (Create React App):

1. `src/App.js` — main layout: upload panel + Q&A panel
2. `src/components/FileUpload.js` — drag-and-drop or click-to-browse;
POST multipart to `/upload`;
show progress bar + success toast
3. `src/components/QuestionBox.js` — text input + submit button;
POST {question} to `/ask`;
display answer in a card
4. `src/api.js` — axios instance pointing to `localhost:8000`

Constraints:

- Plain CSS only — no Tailwind, no Material UI
- All API calls go through src/api.js — no inline fetch
- Show a spinner during API calls; show an error banner on failure
- Mobile-responsive layout (flexbox, single column on <768 px)

Success criteria: user can upload a PDF and ask a question entirely within the browser with no console errors.

💡 Why This Works

Naming each component file and its responsibility removes all guesswork. The 'no inline fetch' constraint enforces a clean separation between UI and data-fetching from the very first commit, making the code easier to maintain.

🔑 Key Instructions Explained

- Name every component + its file — prevents a single 500-line App.js
- Specify the API base URL — avoids CORS bugs on first run
- Enumerate UI states (spinner, error) — completeness check for the model
- Responsive constraint — prevents a desktop-only layout

PROMPT 05 — UX Enhancements — Feedback & History

Optimized Prompt

You are a senior frontend engineer updating the React app from Prompt 04.

Add two features:

1. Response Feedback

- Thumbs-up / thumbs-down buttons below each answer card
- POST {question, answer, rating: 'up' | 'down'} to /feedback
- Disable buttons after one vote; show 'Thanks for your feedback!'

2. Conversation History

- src/components/History.js — scrollable list of past Q&A pairs
- Most recent entry at the top
- Each entry shows the question, answer, and feedback rating (if given)
- Persist history in localStorage so it survives page refresh

Constraints:

- Do NOT re-engineer existing components — extend them only
- History state lives in App.js and is passed down as props
- No new npm packages

Success criteria: after three Q&A interactions the history panel shows all three pairs in reverse-chronological order, and ratings are preserved after a full page refresh.

Why This Works

Incrementally adding features to existing components is much safer than asking the model to rewrite the app. Placing history state in App.js avoids prop-drilling chaos and keeps the architecture predictable for beginners learning React.

Key Instructions Explained

- Specify the feedback payload schema — backend team can stub /feedback immediately
- Reverse-chronological order stated explicitly — avoids a common UX bug
- localStorage persistence in the prompt — ensures offline-friendly behavior
- No new packages constraint — keeps the bundle lean

Prompt Engineering Tips for RAG Projects

1 — Always Include Prior Context

Each prompt references what was built in previous steps. This prevents the model from regenerating files and keeps the codebase coherent across sessions.

2 — Specify Constraints Explicitly

"Do NOT use LangChain RetrievalQA" or "Plain CSS only" are not restrictions — they are guardrails that keep the implementation readable and teachable for beginners.

3 — Include Success Criteria

A clear pass/fail condition (e.g., "a question about an unrelated topic returns 'I don't know'") lets the model self-evaluate and often produces better test coverage automatically.

4 — Name Every File

Naming each output file prevents the model from choosing convenient (but inconsistent) file names. It also makes code review and iteration much faster.

5 — Embed Literal Strings Where It Matters

For the LLM system prompt and error messages, copy the exact text into the engineering prompt. The model will reproduce it verbatim in the code — no paraphrasing, no drift.

☑ Built with these 5 prompts you will have a fully working RAG system — from a blank folder to a React app that answers questions grounded in your documents. Fork, adapt, and share. Happy building!